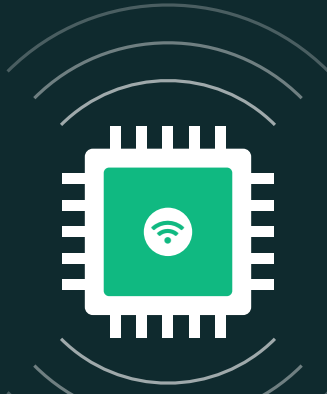




# Réseaux IoT

FiPy ■ MQTT ■ Grafana

R4.IOM.09 — Acquisition à supervision

## Équipe

GRONDIN Angélique  
HONORINE Kylian  
GRONDIN Benjamin

## Encadrant

MOURAD Nour

## ☰ Sommaire

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Objectifs</b>                                   | <b>1</b> |
| <b>2</b> | <b>Architecture du système</b>                     | <b>1</b> |
| <b>3</b> | <b>Protocoles utilisés</b>                         | <b>1</b> |
| 3.1      | USB — alimentation et débogage . . . . .           | 1        |
| 3.2      | Wi-Fi — transport TCP/IP . . . . .                 | 2        |
| 3.3      | MQTT — publication/abonnement . . . . .            | 2        |
| <b>4</b> | <b>Capteurs et actionneur</b>                      | <b>2</b> |
| 4.1      | Capteurs PySense . . . . .                         | 2        |
| 4.2      | Actionneur — LED RGB . . . . .                     | 2        |
| <b>5</b> | <b>Format JSON et flux de publication</b>          | <b>2</b> |
| <b>6</b> | <b>Stockage MySQL et Grafana</b>                   | <b>3</b> |
| 6.1      | Base de données <code>iot.mesures</code> . . . . . | 3        |
| 6.2      | Requête type Grafana . . . . .                     | 3        |
| 6.3      | Dashboards . . . . .                               | 3        |
| <b>7</b> | <b>Performances et énergie</b>                     | <b>4</b> |
| <b>8</b> | <b>Difficultés rencontrées</b>                     | <b>4</b> |
| <b>9</b> | <b>Conclusion</b>                                  | <b>4</b> |

## 1 Objectifs

### 🎯 Mission

Déployer une chaîne IoT complète — de la mesure physique à la visualisation — mobilisant un microcontrôleur Pycom FiPy, des capteurs embarqués PySense, le protocole MQTT, et un dashboard Grafana temps réel.

Quatre objectifs structurent ce TP :

1. Configurer un réseau sans fil local FiPy → broker MQTT
2. Acquérir les mesures des capteurs et les sérialiser en JSON
3. Afficher les données en temps réel sur Grafana
4. Intégrer un retour visuel via la LED RGB embarquée

## 2 Architecture du système

## Chaîne de traitement

Capteurs PySense → FiPy (ESP32) → Wi-Fi (smartphone) → Mosquitto (PC) → Grafana

| Composant    | Rôle                                                                 |
|--------------|----------------------------------------------------------------------|
| PySense      | Collecte des grandeurs physiques via I <sup>2</sup> C                |
| FiPy (ESP32) | Microcontrôleur — logique embarquée, conversion, publication MQTT    |
| PC           | Environnement de développement + hébergement de Mosquitto            |
| Smartphone   | Passerelle Wi-Fi pour contourner les restrictions réseau pédagogique |
| Mosquitto    | Broker MQTT : centralise les messages et les redirige par topics     |
| Grafana      | Visualisation dynamique des séries temporelles                       |

Cette architecture en couches respecte le principe IoT fondamental : **collecter, transmettre, traiter, visualiser, agir**.

## 3 Protocoles utilisés

### 3.1 USB — alimentation et débogage

La liaison USB entre FiPy et PC remplit deux fonctions : alimentation 5 V et communication série pour les logs MicroPython via l'extension PyMakr de VS Code.

### 3.2 Wi-Fi — transport TCP/IP

Le FiPy est configuré en mode **station (STA)**. Il rejoint le point d'accès du smartphone, obtient une IP par DHCP, puis ouvre une connexion TCP vers le port **1883** (MQTT en clair) du broker. Un mécanisme de reconnexion automatique gère les pertes momentanées.

### 3.3 MQTT — publication/abonnement

MQTT est le standard de fait pour l'IoT : faible empreinte, architecture pub/sub, résilience aux pertes de connexion. Le FiPy publie sur des topics dédiés :

```
/iot/groupe2gh/temperature
/iot/groupe2gh/humidite
/iot/groupe2gh/luminosite
/iot/groupe2gh/altitude
```

Le broker Mosquitto centralise les messages puis les redirige vers les clients abonnés (ici Grafana).

## 4 Capteurs et actionneur

### 4.1 Capteurs PySense

Tous interfacés via le bus I<sup>2</sup>C (SDA/SCL), 2 fils suffisent.

| Capteur     | Mesure                    | Type             | Précision   | Fréq.  |
|-------------|---------------------------|------------------|-------------|--------|
| BME280      | T° / humidité / pression  | I <sup>2</sup> C | ±1°C, ±3%HR | 0,5 Hz |
| LTR329ALS01 | Luminosité                | I <sup>2</sup> C | ±5%         | 0,5 Hz |
| (calculée)  | Altitude (issue pression) | calc.            | ±1 m        | 0,5 Hz |

## 4.2 Actionneur — LED RGB

La LED intégrée du FiPy joue le rôle d'indicateur d'état :

- ● **Bleu** — température inférieure à 25 °C
- ● **Violet** — température supérieure à 25 °C
- ● **Rouge** — perte de connexion Wi-Fi

## 5 Format JSON et flux de publication

Chaque acquisition est sérialisée en JSON avant publication :

```
{
  "temperature": 30.03,
  "humidite": 64.0,
  "luminosite": 882,
  "altitude": 10.2
}
```

Cinq étapes structurent la boucle principale en MicroPython :

1. **Initialisation** — modules `network`, `pysense`, `SI7006A20`, `LTR329ALS01`, `pycom`, `umqtt`
2. **Connexion Wi-Fi** — mode STA, attente DHCP
3. **Acquisition** — lecture cyclique des capteurs (toutes les 2s)
4. **Publication MQTT** — envoi sur les topics dédiés, gestion des reconnections
5. **Indication LED** — mise à jour de la couleur selon les seuils

## 6 Stockage MySQL et Grafana

### 6.1 Base de données `iot.mesures`

Une table relationnelle persiste les acquisitions au-delà du flux MQTT :

| Colonne                  | Description                                    |
|--------------------------|------------------------------------------------|
| <code>ts_epoch</code>    | Timestamp UNIX de la mesure                    |
| <code>t_c</code>         | Température en degrés Celsius                  |
| <code>h_pct</code>       | Humidité en pourcentage                        |
| <code>lux_ch0/ch1</code> | Luminosité sur deux canaux                     |
| <code>alt_m</code>       | Altitude calculée en mètres                    |
| <code>src_device</code>  | Identifiant émetteur (ex. <code>fipy1</code> ) |
| <code>ts_ingest</code>   | Horodatage lisible de l'insertion              |

## 6.2 Requête type Grafana

```
SELECT
  FROM_UNIXTIME(ts_epoch) AS time,
  t_c AS value,
  src_device AS metric
FROM mesures
ORDER BY ts_epoch;
```

La fonction `FROM_UNIXTIME` convertit le timestamp en format temporel utilisable par Grafana, qui interprète alors les données comme une série temporelle.

## 6.3 Dashboards

Quatre panels **Gauge** avec seuils colorés affichent la dernière valeur reçue :

- **Vert** — valeur dans la plage normale
- **Orange** — valeur de vigilance
- **Rouge** — valeur critique

Rafraîchissement à 5 s : bon compromis entre réactivité et stabilité du rendu.

## 7 Performances et énergie

| Métrique                  | Valeur observée         |
|---------------------------|-------------------------|
| Période d'échantillonnage | 2 secondes              |
| Débit messages            | ~30 / minute            |
| Taille payload JSON       | ~80 octets              |
| Latence FiPy → Grafana    | ~400 ms                 |
| Consommation FiPy seul    | 120 mA @ 3,3 V (~0,4 W) |
| PySense + capteurs        | +30 mA                  |
| LED RGB                   | +5 à 10 mA              |
| Total système             | ~0,5 W en régime normal |

La faible empreinte mémoire de MicroPython et l'efficacité du header MQTT (contre HTTP) permettent une exécution stable sur plusieurs heures sans fuite ni blocage.

## 8 Difficultés rencontrées

| Problème                                       | Solution                                               |
|------------------------------------------------|--------------------------------------------------------|
| Bornes Eduroam/RT-WiFi bloquent les ports MQTT | Passerelle smartphone en mode point d'accès            |
| Incompatibilité de versions des libs Pycom     | Mise à jour firmware FiPy et PySense avant déploiement |
| Mapping JSON ↔ topics Grafana                  | Format de message uniformisé pour tous les capteurs    |

## 9 Conclusion

Ce TP matérialise la boucle IoT canonique : capter, transmettre, traiter, visualiser, agir. La maîtrise simultanée du firmware embarqué (MicroPython), du transport (MQTT/Wi-Fi), du stockage (MySQL) et de la visualisation (Grafana) constitue un socle solide pour des projets plus ambitieux — LoRa, LTE-M, architectures cloud distribuées.

L'intégration d'une commande MQTT inverse (bouton Grafana → LED FiPy) symbolise la boucle fermée : la donnée ne se contente plus d'être observée, elle devient un levier d'action.

*« Internet of Things : connecter le monde physique au monde numérique. »*